

Section 1: Objectives & Success Criteria

Context

Ferrite is a developer-focused Rust browser built on Servo that differentiates through capability-based agent governance. This section defines what we're building, for whom, what counts as success, and what is explicitly out of scope.

Problem Statement

Agentic browsers (Perplexity Comet, ChatGPT Atlas, Dia) give AI agents the same access as authenticated users — full DOM access, network capability, storage access, credential use — with no formal governance model. When these agents are manipulated via prompt injection (hidden malicious instructions on webpages), they can exfiltrate data, make unauthorized purchases, or modify user accounts. OpenAI has acknowledged that prompt injection "may never be fully solved" at the model level.

The fundamental gap: no browser provides capability-mediated governance of agent actions with cryptographic verifiability. All current defenses are model-level (adversarial training, classifiers, user confirmation prompts). No architectural enforcement exists.

Target Audience: Three Personas

Persona 1: "Agent Alex" — AI/Agent Developer (PRIMARY)

Who they are: Builds browser automation workflows using Playwright, Stagehand, Anthropic Computer Use, or custom LLM agents. Deploys agents that browse autonomously — filling forms, comparing prices, booking travel, triaging emails.

Pain points with existing tools:

- No way to audit what an agent did and why during a session
- No way to prove an agent didn't perform unauthorized actions
- No sandbox to test agent behavior against adversarial web content before production
- No per-action governance — agents either have full access or no access
- When prompt injection happens, there's no blast radius containment

What Ferrite gives Alex:

- Capability tokens that restrict agent access to specific origins, DOM elements, and network targets
- Per-action risk classification with configurable approval workflows
- Session recording and replay via the audit log
- Adversarial testing tools (inject prompt injection payloads, verify governance blocks them)
- Cryptographic proof that the agent only performed authorized actions

Persona 2: "Pentester Pete" — Security Researcher (SECONDARY)

Who they are: Analyzes browser extensions, tests prompt injection attacks, evaluates agentic browser security posture.

Pain points:

- Chrome DevTools cannot inspect extension cross-tab communication or content script injection patterns
- No visibility into what capabilities extensions actually exercise at runtime
- No forensic-grade audit trail for post-incident analysis

What Ferrite gives Pete:

- Capability inspector showing every active token, its scope, and the policy rule that granted it
- Audit log with cryptographic verification for forensic analysis
- Extension sandbox with deterministic replay

Persona 3: "Private Priya" — Privacy-Conscious Developer (TERTIARY)

Who they are: Wants Brave-level privacy with developer-grade transparency.

What Ferrite gives Priya:

- adblock-rust content blocking out of the box (every block logged and inspectable)
 - No mandatory account, local-first data
 - Auditable, transparent browser behavior by design
-

Deliverables (8 Months)

Deliverable 1: Working Browser Prototype

A Servo-based browser with:

- Tab management, navigation, address bar (basic but functional)
- Capability broker mediating all extension and agent actions
- Wasm-sandboxed extension runtime with 7+ capability types
- Agent governance with risk classification and step-up consent
- Hash-chained audit log (upgraded to Merkle tree by month 5)
- adblock-rust content blocking
- Accessibility tree extraction via broker
- Polished dark-mode Iced UI with custom DevTools panels
- CEF comparison harness (Track B) demonstrating engine-agnostic architecture

Deliverable 2: Two Standalone Developer Tools

- **Extension Security Analyzer (CLI):** Loads a `.wasm` extension, enumerates its requested capabilities, evaluates them against a policy file, reports violations and over-permissions. Usable without the browser.
- **Agent Action Inspector (GUI):** Standalone Iced application that reads audit log files, displays an interactive timeline of agent actions, shows capability grants/denials, verifies hash chain / Merkle proof integrity. Usable without the browser.

Deliverable 3: Research Paper

- Targeting NDSS 2027 or ACSAC 2026
 - Documents the architecture, implementation, and evaluation
 - Secondary priority — the browser is the product; the paper documents it
-

Success Criteria (Quantitative)

Criterion	Target	How Measured
Capability enforcement completeness	100% — no privileged action executes without a valid capability token	Automated test suite that attempts every action type without tokens; all must fail
Audit coverage	Every agent and extension action has an audit entry	Differential test: run agent workflow, count actions vs audit entries; must be 1:1
Audit integrity	Hash chain is unbroken; Merkle proofs verify	Automated integrity checker run after every test suite
Adversarial defense	Governance blocks unauthorized actions in 20+ adversarial scenarios	Test corpus including indirect prompt injection, privilege escalation, scope escape
Broker latency	<1ms median, <5ms p99	Microbenchmark: 10,000 token validations, measure latency distribution
Extension sandbox isolation	Extensions cannot access capabilities beyond their manifest	Sandbox escape test suite: extension attempts unauthorized host function calls
Content blocking efficacy	≥10% bandwidth reduction on tracker-heavy test corpus	Measure bytes transferred with and without adblock-rust on 50-URL corpus
Engine agnosticism	Capability broker works identically on Servo (Track A) and CEF (Track B)	Run identical test suite on both engines; results must match

Success Criteria (Qualitative)

- A new team member reading this reference document set can understand the full system in one sitting
- The three demos (extension enforcement, agent governance, Servo vs CEF) each run in under 2 minutes and clearly show the value proposition
- The extension security analyzer and agent action inspector are installable and usable by someone who has never seen the browser

Explicit Non-Goals (Out of Scope)

These items are intentionally excluded. Do not implement them. Do not spend time designing them. Document them as future work.

1. **General-purpose web compatibility.** Ferrite does not need to render Twitter, Gmail, or complex SPAs correctly. Servo's ~79% WPT pass rate is acceptable. Design demos around simple, Servo-compatible pages.
2. **Windows/macOS native builds within the 8-month scope.** Linux-first. Windows is a stretch goal for month 7-8 only.
3. **Custom JavaScript engine.** SpiderMonkey via Servo is the JS engine. Do not attempt to replace or abstract it.
4. **Standard DevTools (DOM inspector, console, network waterfall).** Use Firefox remote debugging for these. Build only the differentiating panels.
5. **Full WebExtensions API compatibility.** Ferrite extensions are Wasm modules with capability declarations. MV3/WebExtensions compatibility is future work.
6. **Energy-aware scheduling, differential rendering, offline-first substrate, privacy-partitioned state.** These are documented as future work. Do not start them unless all committed modules are complete and polished.
7. **Public transparency log integration (Sigstore Rekor).** The audit log is local-only. Public transparency is future work.
8. **Formal verification of capability token properties.** Desirable for the paper but not required for the prototype.
9. **Mobile (Android/iOS) support.**
10. **User account system, sync, or cloud features.** Ferrite is local-first.